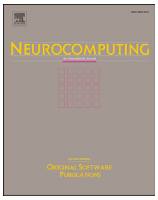
Neurocomputing 699 (2026) 134438



Contents lists available at ScienceDirect

Neurocomputing

journal homepage: www.elsevier.com/locate/neucom



MeMAT: Multi-agent transformer with deep long-term memory, short-term memory, and persistent memory

Gege Sun*, Weiqiang Jin, Yu Zhang, Guizhong Liu*

School of Information and Communications Engineering, Xi'an Jiaotong University, Innovation Harbour, Xi'an 710049, Shaanxi, China

HIGHLIGHTS

- MeMAT: a novel multi-agent Transformer integrating three memory mechanisms for enhanced decision-making.
- Develops three MeMAT variants (MeCMAT, MeGMAT, MeLMAT) to explore different memory interaction paradigms.
- Identifies MeCMAT as the most effective variant, with a contextual memory design that yields clear performance gains over strong baselines.
- Ablation studies establish the critical role of structured memory in long-horizon reasoning.

ARTICLE INFO

Communicated by J. Lian

Keywords:
Multi-agent reinforcement learning
Memory transformer
Deep long-term memory
Persistent memory
Memory interaction mechanisms

ABSTRACT

Learning in partially observable multi-agent systems remains challenging because long-term interactions and coordinated behaviors are difficult to model. Traditional multi-agent reinforcement learning (MARL) methods often rely on limited memory or simple recurrent networks and struggle to capture complex temporal dependencies. To address this limitation, this study proposes the Multi-Agent Transformer with Memory (MeMAT), the first framework to integrate deep neural long-term memory into online MARL. Unlike prior work that relies on either compressed vector-valued memory (RNNs) or uncompressed matrix-valued memory (Transformers), MeMAT introduces a different memory paradigm: a deep MLP whose parameters encode compressed historical experiences via surprise-driven updates, combined with persistent memory for task priors and short-term memory for immediate coordination. Three memory interaction variants are investigated: Multi-Agent Transformer with Contextual Memory (MeCMAT), Multi-Agent Transformer with Gated Memory (MeGMAT), and Multi-Agent Transformer with Memory Layer (MeLMAT), each implementing a distinct memory addressing strategy. Extensive experiments on the StarCraft Multi-Agent Challenge and Google Research Football benchmarks show that MeMAT outperforms strong baselines, especially in Super Hard scenarios. Ablation studies further indicate that the three memory components play complementary roles in long-term decision-making. The implementation of MeMAT is publicly available at https://github.com/suneexxhh/MeMAT.

1. Introduction

Multi-agent reinforcement learning (MARL) algorithms [1–4] play a crucial role in solving complex sequential decision-making tasks by modeling interactions among multiple agents and their shared environment. This paradigm has found broad applications across various domains, including autonomous driving [5–9], traffic management [10–13], unmanned aerial vehicle control [14–18], and gaming systems [19–21]. Partial observability [22,23] remains a fundamental and long-standing challenge in MARL, as each agent can perceive only entity information within its own field of view rather than accessing the complete global state. This constraint severely hinders agents from establishing effective long-term coordination. To address this issue, memory-augmented MARL frameworks offer a promising direction by providing dedicated memory for storing and recalling past experiences, thereby enhancing cooperative decision-making.

Memory mechanisms play a pivotal role in MARL [24–27], especially under partial observability [28], dynamic environments, and complex agent interactions [29,30]. Incorporating explicit memory structures enables agents to effectively store, retrieve, and reason about past experiences, according to the concept of "working memory" in cognitive science [31,32]. This capability is crucial for informed decision-making

* Corresponding authors.
Email addresses: sunee@stu.xjtu.edu.cn (G. Sun), liugz@xjtu.edu.cn (G. Liu).

https://doi.org/10.1016/j.neucom.2026.134438
Received 9 December 2025; Received in revised form 12 May 2026; Accepted 3 July 2026
Available online 4 July 2026
0925-2312/© 2026 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

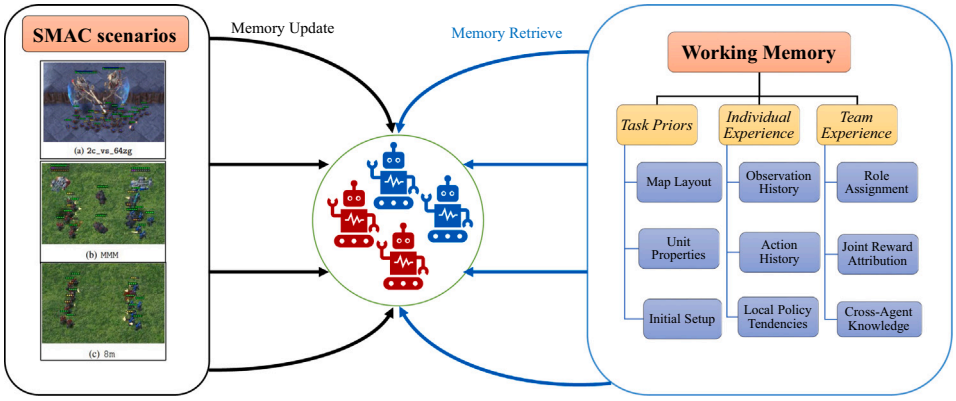


Fig. 1. Illustration of how agents use the memory to guide strategy in SMAC scenarios.

and mirrors human cognition, where individuals leverage accumulated experiences to produce adaptive behaviors in novel situations. Effective memory modules not only empower agents to recall their own observation histories but also allow them to model, predict, and adaptively respond to the behaviors of other agents in MARL. As shown in Fig. 1, agents in StarCraft Multi-Agent Challenge (SMAC) scenarios retrieve information from a shared working memory that integrates task priors, individual experiences, and team experiences, enabling coordinated decision-making.

Despite this central role of memory in MARL, designing suitable and efficient memory structures tailored specifically for MARL remains a prominent challenge and active research direction. MARL methods typically incorporate memory mechanisms to augment local observations by capturing temporal dependencies in decision-making. Most existing approaches [1,2,25,33] rely on RNNs (e.g., LSTMs, GRUs) to compress history into a fixed-size vector-valued state. While this enables efficient sequential modeling, a very long context cannot be properly captured in such a compact representation, fundamentally limiting the capacity for long-term dependencies [16,34]. In parallel, several approaches have extended memory-augmented architectures by integrating transformer-based models and explicit working memory modules (e.g., slot-based, content-addressable memories) [2,30,35], improving agents' abilities to cooperate, coordinate, and generalize across tasks. Nevertheless, transformer-based models maintain historical data as uncompressed, matrix-valued memory within a fixed context window and therefore suffer from quadratic time and memory complexity [36]. Furthermore, the memory remains shallow and short-term [37], which lacks the ability to compress and retain knowledge over long horizons. This limitation is particularly problematic for multi-agent credit assignment [38,39], where agents must retrospectively attribute delayed team rewards to past actions across both spatial and temporal dimensions under partial observability [40].

To address the limitations of existing memory approaches, Behrouz et al. [36] recently proposed a neural long-term memory. As shown in Fig. 2, unlike vector-valued hidden states or matrix-valued context windows, Titans' long-term memory is implemented as a deep neural network (e.g., MLPs) that can be updated and queried, acting as a meta in-context learner to capture long-term dependencies more effectively. In addition, they present Titans and its three variants, in which the long-term memory is incorporated as: (i) a context, (ii) a layer, and (iii) a gated branch. Although Titans has demonstrated impressive performance in domains such as language modeling and reasoning through adaptive neural memory, it has not yet been explored or evaluated in MARL. This absence is notable because MARL requires memory mechanisms that can support decentralized action generation, inter-agent dependency modeling, and coordination under partial observability. Titans meets this demand by offering a principled way to integrate task priors, historical experiences, and cooperative strategies in MARL. Consequently, it is well-suited to tackle core MARL challenges such as partial observability, long-term credit assignment, and non-stationarity [1].

Inspired by this, this paper proposes a novel framework for MARL under partial observability and introduces **Multi-Agent Transformer with Memory (MeMAT)**. MeMAT is an encoder–decoder framework in which the memory system captures multi-scale MARL information: Persistent Memory encodes global task priors, long-term memory stores compressed historical experiences, and the Short-Term Memory dynamically models immediate coordinated interactions among agents. A key question is how to best integrate this long-term memory. To this end, three variants are introduced and evaluated: (1) Multi-Agent Transformer

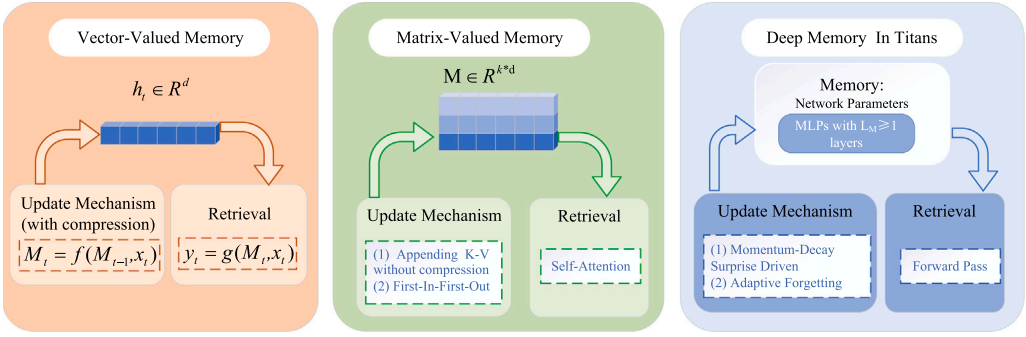


Fig. 2. Deep neural memory in Titans versus vector-valued and matrix-valued memory.

with Contextual Memory (MeCMAT), which treats the long-term memory output as an additional context that is concatenated with the input sequence for the attention module; (2) Multi-Agent Transformer with Gated Memory (MeGMAT), which uses the long-term memory to generate a gating vector that modulates the intermediate representations within the network; and (3) Multi-Agent Transformer with Memory Layer (MeLMAT), which incorporates the long-term memory as a standalone, parallel processing layer whose output is concatenated with the observation embeddings to form the input to the Transformer encoder. Comprehensive experiments are conducted on SMAC [41] and Google Research Football (GRF) [42]. Results show that MeCMAT achieves superior performance over strong baselines, especially in Super Hard scenarios, demonstrating the structured, multi-scale memory mechanisms enhance long-term coordination capacity in multi-agent systems.

The main contributions are summarized as follows:

- We propose MeMAT, a novel multi-agent Transformer framework with a multi-scale memory architecture, in which long-term, short-term, and persistent memory mechanisms interact through dedicated modules to capture temporal dependencies at different horizons.
- We systematically investigate three memory integration strategies, namely MeCMAT, MeGMAT, and MeLMAT, to analyze how long-term memory can be effectively incorporated into Transformer-based MARL.
- We conduct extensive experiments on SMAC and GRF, showing that MeCMAT performs best among the variants and outperforms representative baselines, with ablations confirming the complementary roles of long-term and persistent memory.

The remainder of this paper is organized as follows. Section 2 provides the formal background. Section 3 reviews related work. Section 4 presents the MeMAT framework and pseudocode. Section 5 reports the experimental results. Finally, Section 6 concludes the paper and outlines future research directions.

2. Preliminaries

This section introduces the cooperative MARL problem (Section 2.1), reviews memory limitations of existing sequence models from a unified read–write perspective (Section 2.2), and presents the Titans neural long-term memory upon which MeMAT is built (Section 2.3).

2.1. Problem formulation

Cooperative MARL problems are often modeled by Markov games $\langle \mathcal{N}, \mathcal{O}, \mathcal{A}, R, P, \gamma \rangle$ [43]. $\mathcal{N} = \{1, 2, \dots, n\}$ is the set of agents. $\mathcal{O} = \prod_{i=1}^{n} \mathcal{O}^i$ is the product of local observation spaces of the agents, namely the joint observation space. $\mathcal{A} = \prod_{i=1}^{n} \mathcal{A}^i$ is the product of the agents' action spaces, namely the joint action space. $R : \mathcal{O} \times \mathcal{A} \to [-R_{\max}, R_{\max}]$ is the joint reward function, $P : \mathcal{O} \times \mathcal{A} \times \mathcal{O} \to \mathbb{R}$ is the transition probability function, and $\gamma \in [0, 1]$ is the discount factor. At time step $t \in \mathbb{N}$, an agent $i \in \mathcal{N}$ observes an observation $o_t^i \in \mathcal{O}^i$ (where $\boldsymbol{o} = (o^1, \dots, o^n)$ is the joint observation) and takes an action a_t^i according to its policy π^i, which is the i-th component of the agents' joint policy $\boldsymbol{\pi}$. At each time step, all agents take actions simultaneously based on their observations with no sequential dependencies. The transition kernel P and the joint policy $\boldsymbol{\pi}$ induce the (improper) marginal observation distribution $\rho_{\boldsymbol{\pi}}(\cdot) \triangleq \sum_{t=0}^{\infty} \gamma^t \Pr(\boldsymbol{o}_t = \boldsymbol{o} \mid \boldsymbol{\pi})$. At the end of each time step, the whole team receives a joint reward $R(\boldsymbol{o}_t, \boldsymbol{a}_t)$ and observes $\boldsymbol{o}_{t+1}$, whose probability distribution is given by $P(\cdot \mid \boldsymbol{o}_t, \boldsymbol{a}_t)$. Following this process over an infinite horizon, the agents aim to maximise the expected discounted cumulative return $J(\boldsymbol{\pi}) \triangleq \mathbb{E}_{\boldsymbol{\pi}} \left[\sum_{t=0}^{\infty} \gamma^t R(\boldsymbol{o}_t, \boldsymbol{a}_t) \right]$.

2.2. A unified memory perspective

This section establishes a unified memory perspective for sequence models. Let $x \in \mathbb{R}^{N \times d_x}$ be the input and M a memory module. Subscripts are used for indexing, with t reserved for time steps.

Memory interaction is formulated through *read* and *write* operations. Reading from a memory matrix $M \in \mathbb{R}^{d_h \times d_h}$ uses a query network q to produce a retrieval result $h_t = M\, q(x_t)$, which is passed to the policy. Writing updates the memory via $M_t = f(M_{t-1}, x_t)$, integrating new information.

From this view, RNNs [34] maintain a hidden state $M_t \in \mathbb{R}^{d_h}$ as memory, with fused read–write dynamics: $M_t = f(M_{t-1}, x_t)$. This enables efficient sequential modeling but compresses all history into a fixed-size vector, fundamentally limiting long-term capacity.

Transformers [37] implement a matrix-valued associative memory $M=(K, V)$, where $Q=xW_Q$, $K=xW_K$, $V=xW_V$ with $W_Q, W_K, W_V \in \mathbb{R}^{d \times d}$; here $K \in \mathbb{R}^{N \times d}$ is the key matrix whose i-th row k_i serves as the retrieval index for token i, and $V \in \mathbb{R}^{N \times d}$ is the value matrix whose i-th row v_i stores the corresponding content to be retrieved. Retrieval uses $\text{Attention}(Q, K, V)=\text{softmax}(QK^\top/\sqrt{d})V$. Unlike RNNs, Transformers write to memory by directly appending (k_t, v_t) at each step *without compression*, so the memory grows with sequence length, incurring quadratic $\mathcal{O}(N^2 d)$ complexity and yielding only shallow, uncompressed representations of history [36].

The fact that RNNs compress history too aggressively whereas Transformers do not compress it at all motivates the deep neural long-term memory of Titans.

2.3. Background: titans' neural long-term memory

Titans [36] encodes memory in the *parameters* of an MLP of depth $L_M \geq 1$. At each time step, the MLP is updated via a gradient step on an associative memory loss, compressing new observations into its weights. Retrieval is a frozen forward pass costing $\mathcal{O}(L_M d^2)$, *independent* of history length. By the universal approximation property [44], this design is strictly more expressive than single-layer linear memory. The precise update and retrieval equations are given in Section 4.1.1.

3. Related work

Existing approaches to equipping agents with memory mechanisms in MARL can be broadly categorized into two lines. One is based on vector-valued memory models (e.g., RNNs [34], LSTMs [45], GRUs [46]), and the other is based on matrix-valued memory models (e.g., Transformers [37]), each offering distinct representational capacities and scalability trade-offs.

3.1. Vector-valued memory models in MARL

As summarized in Table 1, vector-valued memory models, particularly GRUs and LSTMs, serve as the backbone for a wide range of popular MARL algorithms. These models compress history into fixed-size hidden states [27] enable agents to reason over recent observations and have demonstrated strong performance in partially observable tasks, particularly within actor–critic frameworks [47,48]. Nevertheless, vector-valued models face inherent scaling challenges in multi-agent systems due to difficulties in capturing high-dimensional interactions and long-term dependencies.

3.2. Matrix-valued memory models in MARL

Matrix-valued memory architectures, particularly Transformers [37], have gained increasing popularity in MARL due to their capacity

Table 1
Representative MARL algorithms utilizing vector-valued memory modules.

Memory Module	Algorithms
GRU	QMIX [1], MAPPO [2], HAPPO [3], COMA [49], QTRAN [50], QPLEX [51], DGN [52], SOP-CG [53], DDFG [54]
LSTM	MA-LSTMDP3 [25], IC3Net [33], IQL [55], VDN [56], G2ANet [57]
RNN	DCG [58], GCS [59]

to capture rich temporal dependencies through self-attention. Unlike recurrent models that compress history into hidden vectors, Transformers maintain matrix-valued memory representations, enabling agents to attend to different parts of their observation history.

Several studies have successfully adapted Transformers to MARL. UPDeT [60] introduces entity-wise tokenization and policy decoupling to generalize across tasks with varying team compositions on a value-based centralized training with decentralized execution (CTDE) backbone, but maintains no dedicated memory module beyond the recurrent hidden state. MAT [4] formulates MARL as a sequence modeling problem via an encoder–decoder Transformer, where the auto-regressive decoder generates each agent's action conditioned on preceding agents' decisions, yielding a monotonic joint-policy improvement guarantee; however, its context is bounded by the current episode window and lacks dedicated long-range compression. ATM [30] augments agents with a fixed-capacity, slot-based working memory updated via a first-in-first-out mechanism on the value-based backbone; while it introduces explicit memory, its passive overwriting strategy—without prioritising informative events—leads to instability on challenging maps, and its memory footprint scales linearly with the number of agents, rendering it infeasible in large-scale scenarios. The Multi-Agent Decision Transformer (MADT) [61] studies the paradigm of offline pre-training with online fine-tuning in MARL; because it is an *offline* method, a direct and fair comparison with online algorithms is infeasible.

Nevertheless, even in these successful applications, standard Transformers face critical limitations in MARL: their self-attention mechanism exhibits quadratic complexity with respect to sequence length, and they often struggle to model long-range dependencies [62–64]. While efficient variants like Linear Transformers [65–67] have been developed to reduce computational overhead, they generally fail to achieve competitive performance compared to standard Transformers. Furthermore, a very long context cannot be properly compressed into a small vector-valued or matrix-valued state [36]. Additionally, there is a notable scarcity of memory architectures specifically designed for online MARL settings.

In summary, existing memory approaches in MARL present a dichotomy: vector-valued models (e.g., RNNs) are efficient but capacity-limited, while matrix-valued models (e.g., Transformers) are expressive but computationally costly and often rely on shallow, short-term memory. No existing work combines deep neural long-term memory, persistent task priors, and a principled multi-scale integration strategy within an *online* MARL framework.

Our work aims to bridge this gap by introducing the deep, neural long-term memory of Titans into online MARL and, in line with this research direction, investigate Transformer-based memory modules for partially observable settings, enabling agents to leverage localized, structured multi-scale memory for better coordination and decision-making.

4. Multi-agent transformer with memory

To integrate multi-scale memory mechanisms into MARL, this study proposes the Multi-Agent Transformer with Memory (MeMAT). The framework is inspired by advances in memory-augmented neural networks [36] and is built upon the sequential decision-making paradigm introduced in [4]. Within this paradigm, the Transformer architecture is adopted as a sequence-to-sequence model that maps a sequence of joint observations $(o_1, \dots, o_n)$ to a sequence of corresponding actions $(a_1, \dots, a_n)$. The proposed MeMAT framework adopts an encoder–decoder architecture, as shown in Fig. 3. Its main component is a multi-scale memory system consisting of persistent memory, long-term memory, and short-term memory modules. To investigate effective memory integration strategies, three variants are introduced, which differ primarily in how the long-term memory interacts with the main network. MeCMAT (Contextual Memory) treats the long-term memory output as an additional contextual sequence. MeGMAT (Gated Memory) employs the long-term memory module as a gating mechanism. MeLMAT (Memory Layer) incorporates the long-term memory module as an independent parallel processing layer.

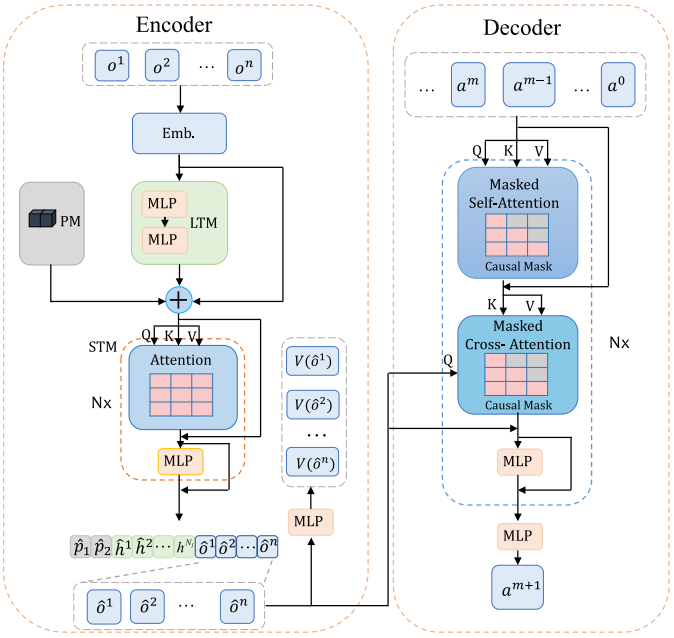


Fig. 3. The encoder–decoder architecture of MeCMAT. At each time step, the encoder processes agents' observations using two complementary memory components: persistent memory (PM) and long-term memory (LTM). Together with short-term memory (STM) realized through self-attention, these modules produce enriched latent representations. The decoder then generates each agent's action in a sequential autoregressive manner using Masked attention.

As shown in Fig. 3, MeCMAT treats the long-term memory as a context sequence for current decision-making. This design enables context-aware memory retrieval, allowing agents to dynamically query relevant historical experiences based on the current situation, much like human decision-making processes that recall past experiences in a context-dependent manner. This approach establishes direct associations between past events and current decisions.

As shown in Fig. 4(a) and Fig. 4(b), two additional variants, namely MeGMAT and MeLMAT, are further investigated, each adopting a different mechanism for memory interaction. In MeGMAT, the long-term memory module functions as a gating mechanism that selectively regulates memory influence by emphasising relevant information while suppressing less informative signals. MeLMAT incorporates the long-term memory module as an additional processing layer, enabling deeper feature transformations for modeling complex memory-function relationships. These complementary architectures facilitate a systematic investigation of how different memory integration strategies influence multi-agent coordination. The details of the three variants are described below.

At the beginning of MeMAT, the raw joint observation at time step t, $\boldsymbol{o}_t = (o_t^1, \dots, o_t^n) \in \mathbb{R}^{n \times d_o}$, is first normalized by a Layer Normalization operation and then mapped into a unified hidden space of dimension d through a linear projection: $\boldsymbol{x}_t = \sigma(\mathrm{LN}(\boldsymbol{o}_t) \cdot \boldsymbol{W}_o)$. Here, $\boldsymbol{W}_o \in \mathbb{R}^{d_o \times d}$, $\sigma(\cdot)$ denotes the GELU activation function, and $\boldsymbol{x}_t \in \mathbb{R}^{n \times d}$. This embedding $\boldsymbol{x}_t$ serves as the input for the subsequent memory and self-attention mechanisms.

4.1. Encoder

We design the encoder to process the embedded joint observations $\boldsymbol{x}_t = (x_t^1, \dots, x_t^n)$ into an integrated representation. By fusing information from long-term, short-term, and persistent memory, the encoder

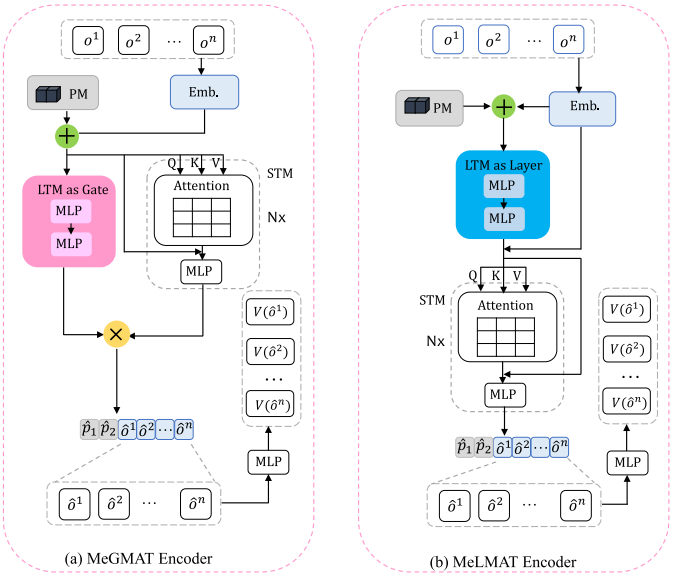


Fig. 4. Comparative encoder architectures. STM, LTM, and PM denote short-term memory, long-term memory, and persistent memory, respectively. (a) MeGMAT employs LTM as a gating mechanism to selectively regulate information flow, while (b) MeLMAT incorporates LTM as an additional processing layer that directly modifies input representations.

produces the final output $\hat{o}_t^{1:n} = (\hat{o}_t^1, \ldots, \hat{o}_t^n)$, which carries a unified representation enriched by the multi-scale memory system. Detailed implementations of these three memory components are presented in Section 4.1.1.

4.1.1. Memory architecture

The MeMAT encoder incorporates three memory components that operate at different temporal scales.

(1) Persistent Memory

The persistent memory consists of learnable tokens that encode stable task priors. These tokens, which are updated during training but fixed during inference, capture high-level knowledge such as spatial structures and strategic patterns. This design provides a consistent global context that improves model generalization.

(2) Long-Term Memory

The long-term memory module is implemented as a deep neural memory based on an MLP with depth $L_M \geq 1$, rather than conventional vector- or matrix-based memory structures. In this work, $L_M = 2$ is adopted, since MLPs with at least two layers are strictly more expressive than single-layer linear memory [44]. The long-term memory module compresses the past information $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_{t-1}$ into its network parameters M_t. The long-term memory operates as an associative memory, learning to map keys to values. For an input $\boldsymbol{x}_t$, it is projected into a key $\boldsymbol{k}_t = \boldsymbol{x}_t W_K^{\text{LTM}}$ and a value $\boldsymbol{v}_t = \boldsymbol{x}_t W_V^{\text{LTM}}$, where $W_K^{\text{LTM}}, W_V^{\text{LTM}} \in \mathbb{R}^{d_{\text{in}} \times d_{\text{in}}}$. The memory module M_t is optimized to minimise the following loss [36]. This objective enables the model to learn associations between keys and values:

$$\ell(M_{t-1}; \boldsymbol{x}_t) = \|M_{t-1}(\boldsymbol{k}_t) - \boldsymbol{v}_t\|_2^2, \tag{1}$$

where $M_{t-1}(\boldsymbol{k}_t)$ denotes the value retrieved from the long-term memory for key $\boldsymbol{k}_t = \boldsymbol{x}_t W_K^{\text{LTM}}$, while $\boldsymbol{v}_t = \boldsymbol{x}_t W_V^{\text{LTM}}$ is the target value. Minimizing Eq. (1) trains M to correctly map each key to its associated value, thereby encoding the input history in the network's weights.

At each time step t, the long-term memory module first performs memory updating and subsequently retrieves contextual information. (i) Memory Update. The long-term memory update mechanism adopts the approach from the Titans [36], which is governed by two biologically-inspired principles: Surprise-driven Memorization and Adaptive Forgetting. Formally, the memory parameters M_t and the surprise state S_t are updated at each step as follows:

$$M_t = (1 - \alpha_t) M_{t-1} + S_t, \tag{2}$$

$$S_t = \eta_t \underbrace{S_{t-1}}_{\text{Past Surprise}} - \theta_t \underbrace{\nabla \ell(M_{t-1}; \boldsymbol{x}_t)}_{\text{Momentary Surprise}}. \tag{3}$$

In Eq. (2), $\alpha_t \in [0, 1]$ is a *forget gate* controlling the rate of decay of past memory, enabling adaptive forgetting of obsolete information. In Eq. (3), $\theta_t \geq 0$ scales the *momentary surprise* $\nabla \ell(M_{t-1}; x_t)$—the gradient of the memory loss, which measures how unexpected the current input is given the current memory state. $\eta_t \in [0, 1]$ is a data-dependent momentum rate controlling the influence of past surprise. All three gates are produced by small learned networks applied to x_t, making the update fully data-dependent.

(ii) Context Retrieval. A forward pass without weight updates retrieves the context for a given query. At time step t, the embedding $\boldsymbol{x}_t$ is projected into a query vector $\boldsymbol{q}_t = \boldsymbol{x}_t W_Q^{\text{LTM}}$. This query is then used to retrieve a relevant context vector from the long-term memory module via content-based addressing:

$$\boldsymbol{h}_t = M_{t-1}^*(\boldsymbol{q}_t). \tag{4}$$

Here, $M^*(\cdot)$ denotes a forward pass through the frozen long-term memory weights (inference without gradient update), retrieving the context most relevant to query $\mathbf{q}_t$. The retrieved vector $\boldsymbol{h}_t \in \mathbb{R}^{n \times d}$ supplies compressed historical context that supplements the current observation.

(3) Short-Term Memory

The short-term memory module is responsible for instantaneous coordinated reasoning across agents. It employs a self-attention mechanism and a multi-layer perceptron (MLP), as well as residual connections to dynamically model not only the information of each agent $m \in \{1, \ldots, n\}$ but also the interrelationships among all agents. This attention is unmasked, meaning it uses a full weight matrix to capture the complete interrelationships among agents.

Specifically, given the embedded queries, keys, and values of n agents: $Q = \{q^1, q^2, \ldots, q^n\}$, $K = \{k^1, k^2, \ldots, k^n\}$, $V = \{v^1, v^2, \ldots, v^n\}$, the attention weights are computed as: $w(q^r, k^j) = \langle q^r, k^j \rangle$, $W = QK^\top \in \mathbb{R}^{n \times n}$. Then, the output representations of agents are obtained by applying the weighted sum of values:

$$\hat{x}^{1:n} = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V, \tag{5}$$

where d_k is the key dimension used to scale the dot-product and prevent vanishing gradients. Eq. (5) computes each agent's updated representation as a weighted sum of all agents' values, with weights determined by pairwise query-key similarity. The use of *unmasked* (full) attention ensures that each agent can directly attend to all others, enabling effective encoding of the global interaction pattern at the current time step.

The output of the short-term memory self-attention, $\hat{\boldsymbol{x}}^{1:n}$, together with the persistent memory tokens and (depending on the variant) the retrieved long-term memory context, forms the encoder input $\boldsymbol{x}_t^{\text{in}}$, which is then processed by the residual MHA–MLP block in Eq. (6).

$$\begin{aligned} \boldsymbol{z}_t &= \boldsymbol{x}_t^{\text{in}} + \text{MHA}\big(\text{LN}(\boldsymbol{x}_t^{\text{in}})\big), \\ \boldsymbol{y}_t &= \boldsymbol{z}_t + \text{MLP}\big(\text{LN}(\boldsymbol{z}_t)\big), \end{aligned} \tag{6}$$

where $\boldsymbol{x}_t^{\text{in}}$ denotes the variant-specific encoder input sequence: for MeCMAT it is $\tilde{\boldsymbol{x}}_t$ as defined in Eq. (7); for MeGMAT and MeLMAT it is defined in Eqs. (9) and (10), respectively. In Eq. (6), LN denotes layer normalization for stable training, residual connections mitigate vanishing gradients, and the MHA block captures multi-head inter-agent interactions.

4.1.2. How to incorporate memory?

Inspired by the Titans architecture [36], we develop three distinct variants of the MeMAT framework to investigate different strategies for memory integration into MARL. Each variant adopts a different mechanism for interacting with the memory components.

(1) MeCMAT

As shown in Fig. 3, MeCMAT incorporates the retrieved long-term memory representation as contextual information for the current observation. This design is motivated by the concept of working memory [31,32,68], in which relevant historical information is actively retrieved to support current decision-making. Specifically, the input to the self-attention module is constructed by concatenating the retrieved memory context h_t (Eq. 4) with the current embedded observations $\boldsymbol{x}_t$ and the persistent memory tokens:

$$\tilde{\boldsymbol{x}}_t = \left[p_1, p_2, \ldots, p_{N_p}\right] \| \boldsymbol{h}_t \| \boldsymbol{x}_t \in \mathbb{R}^{(N_p+N_l+n)\times d}. \tag{7}$$

The resulting representation combines three types of information: current observations representing the immediate situation, retrieved long-term memory context encoding historical information, and persistent memory tokens representing task priors. This composite input is then processed through the self-attention and MLP layers as formulated in Eq. (6) to produce the fused representation $\boldsymbol{y}_t$:

$$\boldsymbol{y}_t = \left[\hat{p}_1, \ldots, \hat{p}_{N_p}\right] \| \hat{\boldsymbol{h}}_t \| \hat{\boldsymbol{o}}_t \in \mathbb{R}^{(N_p+N_l+n)\times d}, \tag{8}$$

where N_p, N_l, and n correspond to the number of persistent memory tokens, long-term memory tokens, and agents, respectively. The final encoder output $\hat{\boldsymbol{o}}_t$ is obtained by slicing the fused sequence $\boldsymbol{y}_t$ and retaining only the segments corresponding to the observation representations of the n agents. This representation serves as the core feature for subsequent decision-making. The complete MeCMAT procedure is summarized in Algorithm 1.

(2) MeGMAT

As shown in Fig. 4(a), MeGMAT employs the long-term memory as a gating mechanism to selectively regulate its influence on the current input and subsequent decisions. This design is motivated by the need to filter potentially noisy or irrelevant historical information, allowing the model to emphasize relevant historical information during decision-making. The input sequence $\boldsymbol{x}_t$ is first concatenated with the persistent memory tokens $\left[p_1, p_2, \ldots, p_{N_p}\right]$. This combined input $\tilde{\boldsymbol{x}}_t$ is processed through the self-attention mechanism of the short-term memory module to capture high-level interrelationships among agents. The output is then gated with the memory output $M(\tilde{\boldsymbol{x}}_t)$ to produce the representation $\boldsymbol{y}_t$:

$$\begin{aligned} \tilde{\boldsymbol{x}}_t &= \left[p_1\ p_2\ \cdots\ p_{N_p}\right] \| \boldsymbol{x}_t, \\ \boldsymbol{y}_t &= \mathrm{Attn}(\tilde{\boldsymbol{x}}_t) \otimes M(\tilde{\boldsymbol{x}}_t). \end{aligned} \tag{9}$$

Here, $\otimes$ denotes a nonlinear gating operation. In the experiments, we normalize the outputs $\boldsymbol{y}_t$ and $M(\tilde{\boldsymbol{x}}_t)$ using learnable vector-valued weights, followed by a non-linearity $\sigma(\cdot)$. Consistent with MeCMAT, the final output $\hat{\boldsymbol{o}}_t$ is obtained by slicing $\boldsymbol{y}_t$. This gating mechanism enables adaptive control over the influence of retrieved memory during decision-making. The complete MeGMAT procedure is provided in Algorithm 2 in Appendix A.

(3) MeLMAT

As shown in Fig. 4(b), MeLMAT treats the long-term memory as a pre-attention enhancement layer that directly modifies input representations, following a conventional layer-wise integration strategy compatible with hybrid architectures. The model begins by augmenting the input sequence $\boldsymbol{x}_t$ with persistent memory tokens $\left[p_1, p_2, \ldots, p_{N_p}\right]$, creating an extended representation. The extended sequence $\tilde{\boldsymbol{x}}_t$ is processed through the long-term memory to obtain memory-augmented features $M(\tilde{\boldsymbol{x}}_t)$, which are then fed into the attention mechanism:

$$\begin{aligned} \tilde{\boldsymbol{x}}_t &= \left[p_1\ p_2\ \cdots\ p_{N_p}\right] \| \boldsymbol{x}_t, \\ \boldsymbol{y}_t &= \mathrm{Attn}\big(M(\tilde{\boldsymbol{x}}_t)\big). \end{aligned} \tag{10}$$

In Eq. (10), the final output $\hat{\mathbf{o}}_t$ is sliced from $\boldsymbol{y}_t$. This sequential design positions the memory module as a dedicated feature enhancement component. This design separates memory augmentation from relational reasoning by performing memory-based feature enhancement before multi-head attention computation. The complete MeLMAT procedure is provided in Algorithm 3 in Appendix A.

4.1.3. Optimization objective

During training, the encoder approximates the value functions by minimizing the empirical Bellman error:

$$\mathcal{L}_{\mathrm{Encoder}}(\phi) = \frac{1}{Tn}\sum_{m=1}^{n}\sum_{t=0}^{T-1}\left[R(o_t, a_t) + \gamma\, V_{\bar{\phi}}\big(\hat{o}_{t+1}^m\big) - V_{\phi}\big(\hat{o}_t^m\big)\right]^2, \tag{11}$$

where $\bar{\phi}$ is the target network's parameter, which is non-differentiable and updated every few epochs.

4.2. Decoder

To maintain the joint action space's tractability and enforce a monotonic improvement guarantee [4], the decoder generates actions auto-regressively for the n agents. To realize this auto-regressive scheme, masked attentions in the decoder capture $a_t^{1:m}$ with triangular matrices where $w(q^r, k^j) = 0$ for $r < j$. Masked self-attention ensures that, when producing the action of the $(m+1)$-th agent, the decoder can only attend to the encoded observations $\hat{o}_t^{1:n}$ and the previously generated actions $a_t^{1:m}$. Consequently, the policy for agent $(m+1)$ is given by $\pi_\theta^{(m+1)}\left(a_t^{(m+1)} \mid \hat{o}_t^{1:n}, a_t^{1:m}\right)$. To optimize the decoder, we adopt the PPO algorithm [69], with the objective defined as

$$\mathcal{L}_{\mathrm{Decoder}}(\theta) = -\frac{1}{Tn}\sum_{m=1}^{n}\sum_{t=0}^{T-1}\min\big(r_t^m(\theta)\,\hat{A}_t,\ \mathrm{clip}(r_t^m(\theta),\, 1\pm\epsilon)\,\hat{A}_t\big), \tag{12}$$

where

$$r_t^m(\theta) = \frac{\pi_\theta^m\left(a_t^m \mid \hat{o}_t^{1:n},\, \hat{a}_t^{1:m-1}\right)}{\pi_{\theta_{\mathrm{old}}}^m\left(a_t^m \mid \hat{o}_t^{1:n},\, \hat{a}_t^{1:m-1}\right)}. \tag{13}$$

Here, $\hat{A}_t$ is an estimate of the joint advantage function computed via Generalized Advantage Estimation (GAE) [70]. Specifically, the joint value baseline is defined as the average of per-agent value estimates produced by the encoder output layer: $\hat{V}_t = \frac{1}{n}\sum_{m=1}^{n} V_\phi\big(\hat{o}_t^m\big)$. The TD residual at step t is then $\delta_t = R(\boldsymbol{o}_t, \boldsymbol{a}_t) + \gamma\,\hat{V}_{t+1} - \hat{V}_t$, and the GAE advantage estimate is $\hat{A}_t = \sum_{l=0}^{T-t-1}(\gamma\lambda)^l\,\delta_{t+l}$, where $\lambda \in [0,1]$ is the GAE decay parameter (see Algorithm 1, line 28).

4.3. Computational complexity

We analyze the per-step computational complexity of MeMAT and compare it with the MAT baseline. (1) The short-term memory self-attention cost is $\mathcal{O}(n^2 d)$, the same as MAT. (2) The long-term memory retrieval and update cost is $\mathcal{O}(L_M d_{\mathrm{in}}^2)$ per step and is independent of the episode length T, unlike full-context Transformer attention with $\mathcal{O}(T^2 d)$ complexity. (3) The persistent memory overhead is small, $\mathcal{O}(N_p n d)$, with $N_p = 2$. The total per-step complexity is $\mathcal{O}(n^2 d + L_M d_{\mathrm{in}}^2)$. In practice, MeCMAT adds about 12% wall-clock training time over MAT on *3s5z_vs_3s6z* using a single RTX 4090.

5. Experiments

The experiments are designed to investigate the following questions:

Algorithm 1 MeCMAT: Multi-agent transformer with contextual memory.

Require: Stepsize α, batch size B, number of agents n, episodes K, steps per episode T

Initialize Encoder ϕ, Decoder θ, Replay buffer $\mathcal{B}$, Long-term Memory $\{M_0\}$, Persistent Memory
for $k = 0, 1, \dots, K-1$ **do**
  for $t = 0, 1, \dots, T-1$ **do**
    Collect observation sequence $o_t^1, \dots, o_t^n$ from environments
    Generate observation embeddings $\boldsymbol{x}_t$
    $\boldsymbol{q}_t \leftarrow \boldsymbol{x}_t W_Q^{\text{LTM}}$
    Retrieve contextual memory: $\boldsymbol{h}_t \leftarrow M_{t-1}^*(\boldsymbol{q}_t)$ (Eq. 4)
    Form encoder input: $\tilde{\boldsymbol{x}}_t = [P_t, h_t, \boldsymbol{x}_t]$ (Eq. 7)
    Generate representations $\hat{o}_t^1, \dots, \hat{o}_t^n$ from encoder
    Feed $\hat{o}_t^1, \dots, \hat{o}_t^n$ into the decoder
    for $m = 0, \dots, n-1$ **do**
      Input $a_t^0, \dots, a_t^m$, infer a_t^{m+1} using auto-regressive decoder
    end for
    Execute joint actions $a_t^0, \dots, a_t^n$ and collect reward $R(\boldsymbol{o}_t, \boldsymbol{a}_t)$
    Store $(\boldsymbol{o}_t, \boldsymbol{a}_t, R(\boldsymbol{o}_t, \boldsymbol{a}_t), M_t)$ in replay buffer $\mathcal{B}$
  end for
  for each training iteration in current episode **do**
    Sample minibatch of B_{mini} steps from $\mathcal{B}$
    for each experience in minibatch **do**
      Generate observation embeddings $\boldsymbol{x}_t$
      Compute long-term memory loss $\mathcal{L}_{\text{LTM}}$ using Eq. (1)
      Compute gradients and update long-term memory parameters
    end for
    Generate $V_\phi(\hat{o}_t^1), \dots, V_\phi(\hat{o}_t^n)$ with encoder output layer
    Compute encoder loss $\mathcal{L}_{\text{Encoder}}(\phi)$ using Eq. (11)
    Compute joint advantage $\hat{A}$ based on $V_\phi(\hat{o}_t^1), \dots, V_\phi(\hat{o}_t^n)$ using GAE
    Input $\hat{o}_t^1, \dots, \hat{o}_t^n$ and $a_t^0, \dots, a_t^n$, generate $\pi_\theta^1, \dots, \pi_\theta^n$
    Compute decoder loss $\mathcal{L}_{\text{Decoder}}(\theta)$ using Eq. (12)
    Update encoder, decoder and persistent memory by minimizing $\mathcal{L}_{\text{Encoder}}(\phi) + \mathcal{L}_{\text{Decoder}}(\theta)$
  end for
end for

- How does the proposed MeMAT framework, particularly the MeCMAT variant, perform compared to state-of-the-art MARL baselines?
- What is the relative effectiveness of different memory interaction mechanisms, including MeCMAT, MeGMAT, and MeLMAT?
- How important are the individual memory components, namely long-term memory, short-term memory, and persistent memory, to the overall performance of MeMAT?

To address these questions, the experimental analysis is conducted in three stages. First, the primary model, MeCMAT, is benchmarked against representative baseline methods. Subsequently, the three MeMAT variants are compared to analyze the impact of different memory integration strategies. Finally, ablation studies are performed to quantify the contribution of each memory component.

5.1. Experimental setup

5.1.1. Environments

StarCraft Multi-Agent Challenge (SMAC) [41] serves as our main benchmark. It has become a standard evaluation suite for cooperative MARL [1–3], comprising a set of StarCraft II micro-management scenarios in which a team of agents must defeat computer-controlled opponents. Scenarios are classified into three difficulty levels—Easy, Hard, and Super Hard—with the game AI set to the highest difficulty (level 7).

Google Research Football (GRF) [42] provides a complementary benchmark. It simulates association football under full standard rules, including corner kicks, fouls, offside, and realistic player physics, and is widely adopted for MARL evaluation [4,19,71,72].

5.1.2. Evaluation protocol

For SMAC, the evaluation metric is the win rate (%), which is defined as the fraction of evaluation episodes in which all enemy units are defeated within the time limit. For GRF, performance is measured by the mean episode score, i.e., the average number of goals scored per episode.

In the SMAC experiments, evaluation is conducted every 10^5 environment steps. Each evaluation consists of 32 independent test episodes executed across 32 parallel threads. In the GRF experiments, evaluation is performed every 25 training iterations, with 32 independent test episodes in each evaluation.

All results are reported as the mean and standard deviation over five independent random seeds. The same set of random seeds is used for both our methods and all re-implemented baselines to ensure a fair comparison.

5.1.3. Implementation and hyper-parameters

All experiments are implemented in PyTorch and executed on a cluster equipped with four NVIDIA GeForce RTX 4090 GPUs using data parallelism.

For baselines, the hyper-parameters recommended in the original studies are adopted for the SMAC and GRF environments. The proposed MeMAT models are built upon the MAT architecture while retaining its core hyper-parameters, including the number of Transformer layers, attention heads, and hidden dimensions. The primary architectural additions are the memory components. The persistent memory module is configured with two memory tokens of dimension 64, while the long-term memory module is implemented as a two-layer multi-layer perceptron with a hidden size of 256. In all experiments, N_l is set equal to the number of agents, denoted by n.

5.2. Comparison with baselines

We compare MeCMAT against six representative online MARL baselines. These methods can be grouped into three categories: recurrent GRU-based approaches (QMIX, MAPPO, and HAPPO), Transformer-based approaches without explicit long-term memory (UPDeT and MAT), and a Transformer-based approach with explicit working memory (ATM). This selection enables the contribution of the proposed multi-scale memory architecture to be evaluated against both conventional recurrent approaches and recent Transformer-based alternatives. The baseline methods are categorised according to learning paradigm and memory mechanism in Table 2.

QMIX [1] is a widely adopted value-decomposition method that monotonically factorizes the joint Q-value into per-agent utilities via a hypernetwork-based mixing network; it has been shown to outperform VDN [56] and COMA [49] on the SMAC benchmark [41], establishing it as a standard CTDE value-based baseline. MAPPO [2] and HAPPO [3] are state-of-the-art policy-gradient methods for homogeneous and heterogeneous settings, respectively. Together they serve as strong recurrent baselines. MAT [4] formulates MARL as a sequence modeling problem via an encoder–decoder Transformer, where the auto-regressive decoder generates each agent's action conditioned on preceding agents' decisions, yielding a monotonic joint-policy improvement guarantee. ATM [30], which augments agents with a fixed-capacity first-in-first-out slot-based working memory on a value-based CTDE backbone, is further included for a rigorous comparison with methods that also employ Transformer-based explicit memory. UPDeT [60] represents the Transformer-backbone, value-based CTDE paradigm, using entity-wise tokenization and policy decoupling to generalize across tasks with varying team sizes. MADT [61] is excluded from direct comparison because

Table 2
Categorisation of baseline methods. *Backbone* denotes the temporal encoder architecture, while *Explicit Memory* indicates whether a dedicated memory module beyond the recurrent hidden state is employed. In the MeCMAT row, STM, LTM, and PM denote short-term memory, long-term memory, and persistent memory, respectively.

Method	Paradigm	Backbone	Memory Mechanism	Explicit Memory
QMIX [1]	Value-based CTDE	GRU (shared)	GRU hidden state	✗
MAPPO [2]	Policy-gradient CTDE	GRU (shared)	GRU hidden state	✗
HAPPO [3]	Policy-gradient CTDE	GRU (separate)	GRU hidden state	✗
UPDeT [60]	Value-based CTDE	Transformer	Self-attention + hidden	✗
ATM [30]	Value-based CTDE	Transformer	First-in-first-out slot memory	✓
MAT [4]	Policy-gradient CTDE	Enc-Dec Transformer	Masked self-attention	✗
MeCMAT	Policy-gradient CTDE	Enc-Dec Transformer	STM + LTM + PM	✓

Table 3
Performance evaluations of win rate and standard deviation on the SMAC benchmark, where UPDeT's official codebase supports several Marine-based tasks only.

Task	Difficulty	MeCMAT	MAT	MAPPO	HAPPO	QMIX	ATM	UPDeT	Steps
3m	Easy	**100.0 ± 0.0**	99.7 ± 0.4	99.7 ± 0.4	96.4 ± 1.7	97.4 ± 0.7	98.7 ± 0.4	97.2 ± 1.4	2e6
1c3s5z	Easy	**100.0 ± 0.0**	98.0 ± 2.2	97.3 ± 1.2	99.5 ± 0.6	95.3 ± 1.0	51.3 ± 42.6	/	2e6
MMM	Easy	**99.0 ± 1.6**	96.0 ± 3.4	77.4 ± 15.2	75.1 ± 15.0	98.6 ± 0.5	93.8 ± 4.5	/	2e6
3s5z	Hard	92.1 ± 6.2	96.7 ± 3.5	60.6 ± 7.7	66.9 ± 22.0	97.8 ± 0.9	**99.1 ± 0.4**	/	5e6
5m vs 6m	Hard	82.0 ± 5.8	57.3 ± 21.0	76.1 ± 4.2	44.5 ± 20.4	67.3 ± 6.5	82.5 ± 3.3	**92.9 ± 6.5**	5e6
8m vs 9m	Hard	**97.7 ± 0.9**	76.4 ± 15.1	91.4 ± 3.8	71.3 ± 17.1	95.5 ± 0.8	90.1 ± 6.6	95.3 ± 0.4	5e6
10m vs 11m	Hard	96.6 ± 2.6	92.1 ± 7.0	76.4 ± 3.3	88.7 ± 5.3	95.2 ± 1.2	92.8 ± 5.3	**98.9 ± 0.8**	4e6
25m	Hard	**100.0 ± 0.0**	99.5 ± 0.7	1.7 ± 2.4	0.3 ± 0.2	95.0 ± 1.4	/	/	3e6
27m vs 30m	Super Hard	**92.7 ± 1.9**	87.6 ± 6.9	0.0 ± 0.0	0.0 ± 0.0	57.4 ± 17.9	/	/	7e6
MMM2	Super Hard	**96.8 ± 0.8**	78.7 ± 22.8	87.7 ± 2.0	35.0 ± 27.6	88.2 ± 2.5	92.3 ± 5.2	/	1e7
6h vs 8z	Super Hard	**93.4 ± 6.9**	79.3 ± 7.8	76.3 ± 6.5	15.0 ± 10.6	8.0 ± 3.7	81.7 ± 6.0	/	1e7
3s5z vs 3s6z	Super Hard	**95.5 ± 3.8**	82.9 ± 11.4	67.6 ± 5.4	26.3 ± 37.2	74.8 ± 3.8	60.2 ± 42.6	/	1.5e7

it is an *offline* MARL method, which makes a fair online performance comparison infeasible.

As reported in Table 3 and Fig. 5, all methods are evaluated by win rate (mean ± standard deviation over five random seeds) on 12 SMAC maps. On Easy maps, all methods converge to near-ceiling win rates, and differences are negligible. As difficulty increases, three divergence patterns emerge: GRU-based methods (MAPPO, HAPPO) collapse on large-scale asymmetric scenarios (e.g., *25m*, *27m_vs_30m*), while QMIX remains competitive on symmetric Hard maps owing to its value-decomposition structure; Transformer-based methods without dedicated memory (MAT, UPDeT) improve over GRU baselines on most Hard maps but plateau on the hardest scenarios; ATM, despite explicit memory, exhibits high variance and is infeasible on large maps due to its agent-count-dependent memory footprint. MeCMAT achieves the highest win rates on the majority of maps, with its advantage growing monotonically with task difficulty, particularly on Super Hard scenarios.

The comparison between MeCMAT and MAT highlights the contribution of the proposed memory architecture beyond the shared Transformer backbone. MAT's performance drops on the most challenging maps, reflecting its inherent limitation: its context is bounded by the episode window and lacks dedicated long-range compression. MeCMAT overcomes this by augmenting the Transformer with long-term memory (which accumulates history via surprise-driven updates) and persistent memory (which supplies stable task-level priors). The consistent improvement validates the complementary roles of these memory modules.

Compared with the slot-based memory mechanism adopted in ATM, the proposed long-term memory design exhibits improved stability and scalability. ATM achieves competitive results on several maps but exhibits high variance and instability on maps like *1c3s5z* and *3s5z_vs_3s6z*. This instability stems from ATM's first-in-first-out slot memory: it passively overwrites old entries without prioritizing informative events. In contrast, MeCMAT's long-term memory is a deep MLP whose parameters continuously encode a compressed, associative representation of the entire history, enabling robust performance on long-horizon tasks. Moreover, ATM's memory scales linearly with the number of agents, making it infeasible on large maps (e.g., *25m*, *27m_vs_30m*) on our 24 GB GPUs. MeCMAT's fixed-size MLP memory is independent of agent count, ensuring scalability.

As shown in Fig. 6, MeCMAT's advantages also extend to the GRF benchmark, where it consistently outperforms baselines across different numbers of agents.

5.3. Evaluation of MeMAT variants

In this section, we compare MeCMAT, MeGMAT, and MeLMAT against MAT to disentangle the effects of different memory integration strategies. Since all models share the same Transformer backbone and memory components (long-term memory and persistent memory), any performance differences are solely attributable to how long-term memory is integrated into the network. Table 4 reports quantitative results on SMAC, and Fig. 7 shows the corresponding training curves.

5.3.1. Performance on SMAC

As shown in Fig. 7 and Table 4, all three variants outperform the MAT baseline in the majority of maps, and the advantage is most consistent on Super Hard scenarios, confirming that the multi-scale memory architecture itself provides a meaningful benefit regardless of the specific integration strategy.

Among the variants, MeCMAT achieves the best win rate on 9 out of 12 maps and demonstrates the highest robustness on Super Hard tasks, with particularly strong results on *27m_vs_30m*, *MMM2*, and *3s5z_vs_3s6z*. This stems from its contextual design, which enables full token-level interaction among long-history context, immediate observations, and task priors within a single attention operation.

Comparing MeGMAT and MeLMAT, neither dominates the other overall: MeGMAT leads on *6h_vs_8z* and *MMM2*, while MeLMAT outperforms on *27m_vs_30m* and *3s5z_vs_3s6z*, yielding broadly comparable aggregate performance. The underlying reasons for these differences are analyzed in Section 5.3.3.

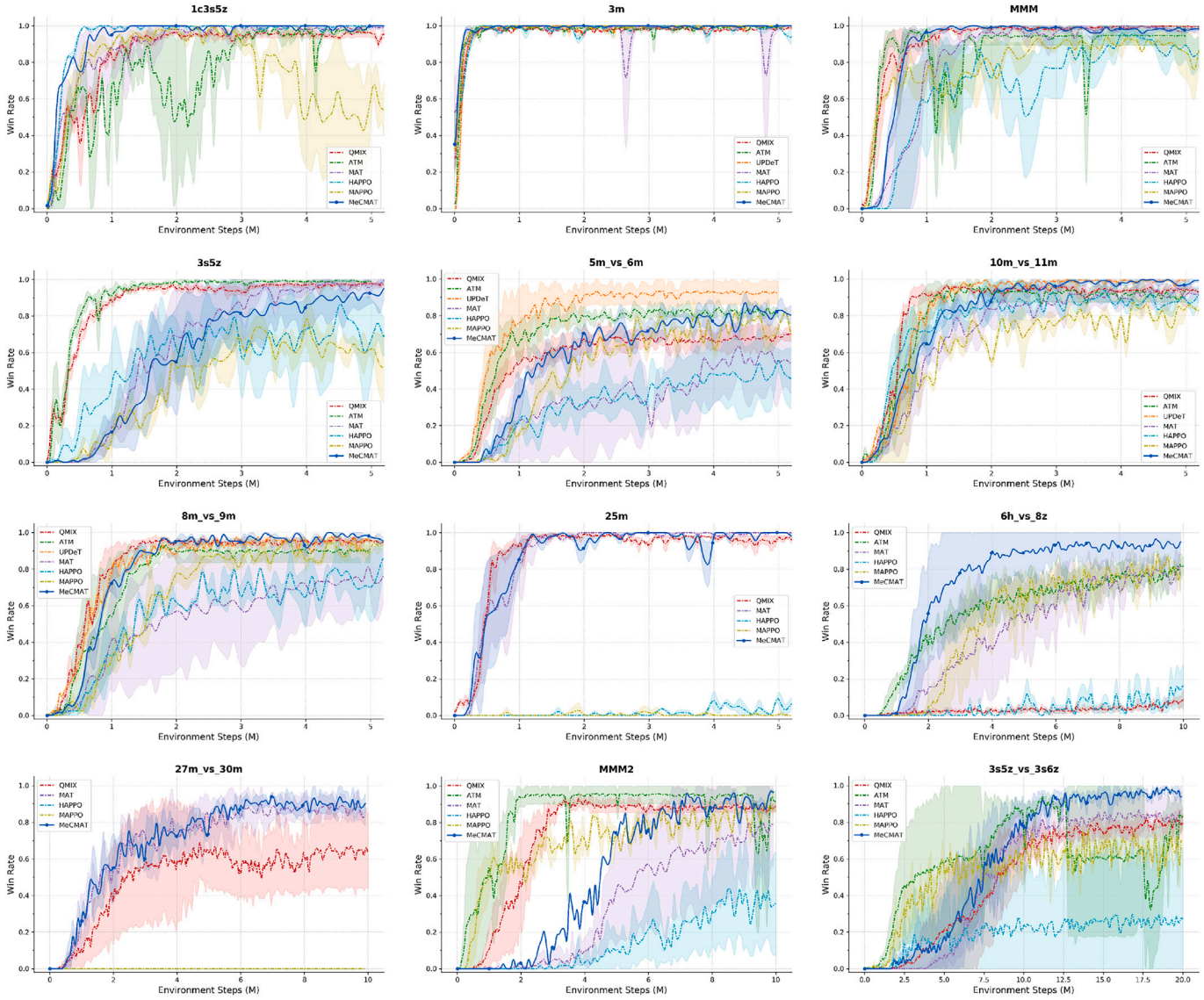


Fig. 5. Win rate curves on 12 maps in SMAC with varying difficulty (Easy, Hard, Super Hard) during training. MeCMAT outperforms the baselines on most maps, with especially notable advantages on the Super Hard maps.

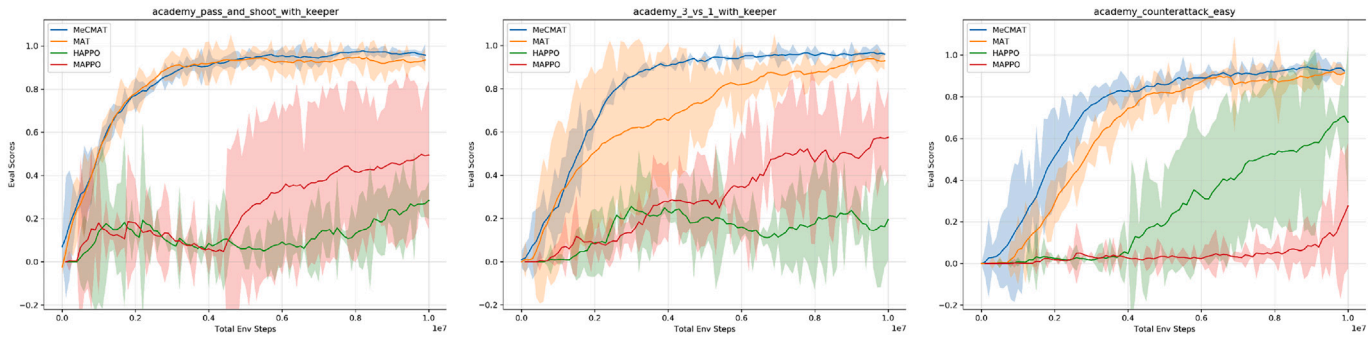


Fig. 6. Comparison of MeCMAT and baselines on GRF tasks with 2–4 agents (from left to right).

5.3.2. *Performance on GRF*

As shown in Fig. 8, the GRF results reveal how agent population influences the effectiveness of different memory mechanisms. In the two-agent scenario, MeGMAT's gated memory accelerates early convergence but limits final performance, while MeCMAT achieves a higher final score. As the number of agents increases to 3 and 4, the superiority

Table 4
Win rate (%) ± std on SMAC: MeMAT variants vs. MAT (mean over seeds). **Bold**: best per task.

Task	Difficulty	MeCMAT	MeGMAT	MeLMAT	MAT	Steps
3m	Easy	**100.0 ± 0.0**	99.4 ± 0.6	99.0 ± 0.0	99.7 ± 0.4	2e6
1c3s5z	Easy	**100.0 ± 0.0**	98.6 ± 1.1	96.7 ± 0.0	98.0 ± 2.2	2e6
MMM	Easy	99.0 ± 1.6	98.8 ± 0.9	**99.2 ± 0.0**	96.0 ± 3.4	2e6
3s5z	Hard	92.1 ± 6.2	93.3 ± 2.3	94.9 ± 0.0	**96.7 ± 3.5**	5e6
5m vs 6m	Hard	**82.0 ± 5.8**	75.2 ± 7.1	72.8 ± 0.0	57.3 ± 21.0	5e6
8m vs 9m	Hard	**97.7 ± 0.9**	93.5 ± 4.0	97.7 ± 1.7	76.4 ± 15.1	5e6
10m vs 11m	Hard	**96.6 ± 2.6**	95.8 ± 2.0	87.3 ± 0.0	92.1 ± 7.0	4e6
25m	Hard	**100.0 ± 0.0**	98.3 ± 1.7	98.9 ± 0.0	99.5 ± 0.7	3e6
27m vs 30m	Super Hard	**92.7 ± 1.9**	78.6 ± 6.2	86.0 ± 1.9	87.6 ± 6.9	7e6
MMM2	Super Hard	**96.8 ± 0.8**	94.2 ± 0.8	88.3 ± 2.2	78.7 ± 22.8	1e7
6h vs 8z	Super Hard	93.4 ± 6.9	**99.2 ± 0.8**	98.5 ± 1.0	79.3 ± 7.8	1e7
3s5z vs 3s6z	Super Hard	**95.5 ± 3.8**	89.4 ± 6.6	94.2 ± 1.9	82.9 ± 11.4	1.5e7

of MeCMAT becomes clear. This demonstrates that its context-based memory retrieval scales more effectively to manage the intricate dependencies and multi-agent interactions in denser environments.

5.3.3. Discussion

The performance differences among MeCMAT, MeGMAT, and MeLMAT suggest an important principle for memory integration in MARL: memory retrieval benefits from token-level interaction mechanisms rather than solely representation-level modulation.

Effectiveness of context-based memory. As illustrated in Fig. 3, MeCMAT allows each agent observation token to directly attend to memory representations through multi-head attention. This design provides several advantages. First, different agents can retrieve different historical information according to their local observations. For example, an agent under threat may retrieve retreat-related experiences, whereas another agent may retrieve information associated with offensive coordination. Second, the attention mechanism facilitates interactions among observation tokens, memory representations, and persistent memory tokens within a unified attention operation, enabling more flexible relational reasoning. Third, memory representations receive direct gradients from the policy objective, allowing the memory module to be optimised jointly with the policy network. By comparison, MeGMAT employs a gating mechanism that applies a global modulation to the

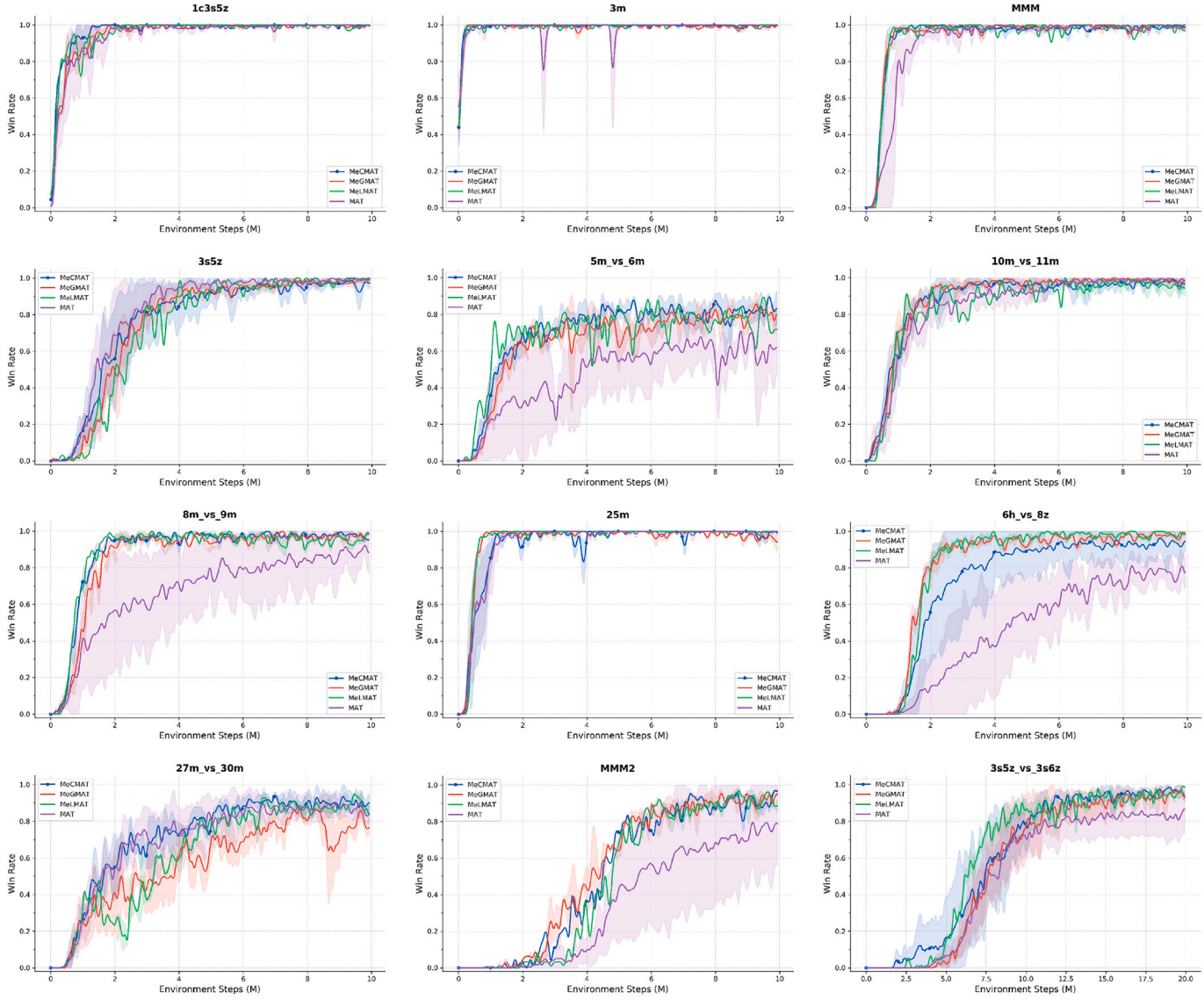


Fig. 7. Win rate curves on 12 maps in SMAC with varying difficulty (Easy, Hard, Super Hard) during training for the three MeMAT variants, MeCMAT, MeGMAT, MeLMAT and the MAT baseline.

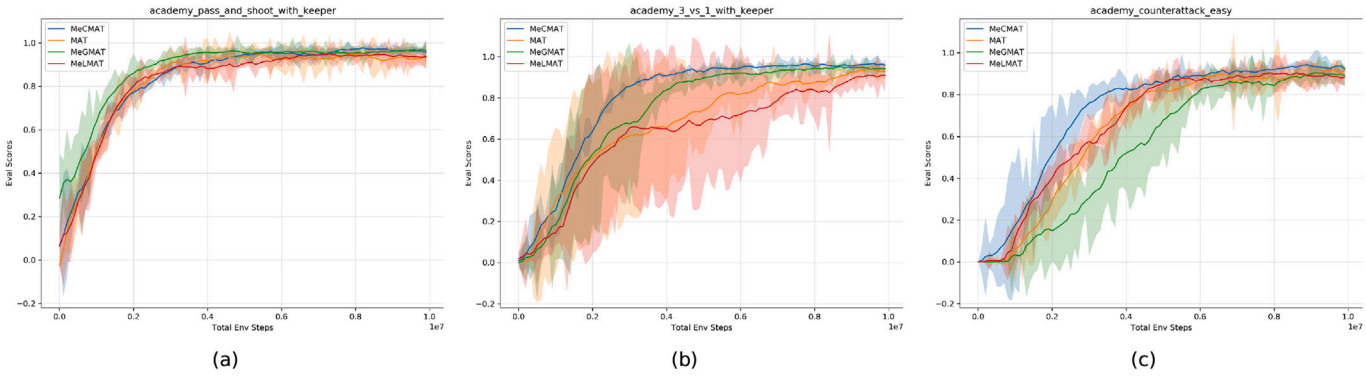


Fig. 8. Performance comparison of the three MeMAT variants, MeCMAT, MeGMAT and MeLMAT and the MAT baseline on GRF tasks with 2–4 agents (from left to right).

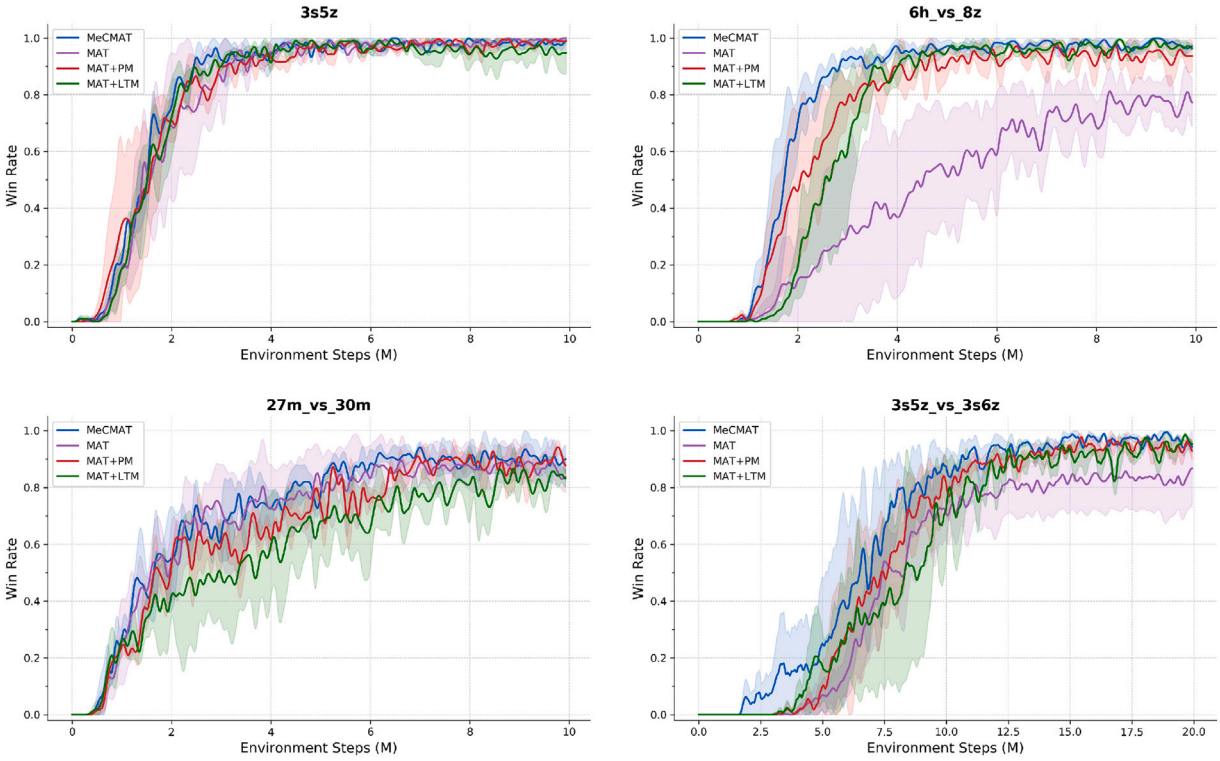


Fig. 9. Performance Ablation on SMAC maps of Varied Difficulties (1 Easy, 3 Super Hard).

intermediate representation, which may limit agent-specific memory retrieval. MeLMAT constructs a fixed memory-augmented representation before relational reasoning takes place, thereby restricting dynamic query-conditioned retrieval during attention computation.

Relation to cognitive memory mechanisms. The MeCMAT architecture is conceptually related to working memory mechanisms studied in cognitive science [31,32]. In this analogy, long-term memory stores compressed historical experiences, working memory retrieves context-relevant information, and attention mechanisms combine current observations with retrieved memory representations. This correspondence may partially explain the strong generalisation capability of MeCMAT across challenging MARL tasks.

Implications for memory integration. The experimental results suggest that different memory integration strategies are suitable for different MARL settings. Context-based memory mechanisms such as MeCMAT are effective for tasks requiring selective and agent-specific memory retrieval. Gated memory mechanisms such as MeGMAT are more suitable when historical information should uniformly influence all agents. Layer-based integration strategies such as MeLMAT are appropriate when memory primarily serves as a feature enhancement component.

5.4. Ablations

Ablation studies are conducted to analyze the contribution of each memory component in MeMAT. The experiments focus on Super Hard scenarios in SMAC. The evaluation compares the baseline MAT against three variants: MAT augmented with long-term memory (MAT + LTM), MAT augmented with persistent memory (MAT + PM), and the complete MeCMAT model incorporating both long-term and persistent memory modules (MAT + LTM + PM).

As shown in Fig. 9, each memory module contributes distinct capabilities to the overall performance: (1) MAT converges slowly and

achieves limited final win rate on all Super Hard maps, confirming that a plain Transformer backbone is insufficient for long-horizon coordination under partial observability. (2) MAT + LTM shows significantly accelerated convergence, particularly on *6h_vs_8z*, where early access to compressed historical experience enables agents to develop effective strategies more quickly. This demonstrates that the surprise-driven long-term memory update is the primary driver of sample efficiency in MeMAT. (3) MAT + PM, while having a slower initial learning phase, achieves more stable and higher final performance in scenarios like *27m_vs_30m*, where task-specific priors provide crucial guidance.

The complete MeCMAT model synergistically combines these benefits, achieving both faster convergence and superior final performance. This demonstrates the complementary nature of long-term memory's dynamic recall and persistent memory's stable knowledge foundation in addressing MARL challenges.

6. Conclusion

This paper presents MeMAT, a framework for integrating structured multi-scale memory mechanisms into Transformer-based MARL. The framework consists of three dedicated components: long-term memory for retaining important historical information through surprise-driven updates, short-term memory for modeling recent agent states and inter-agent interactions via self-attention, and persistent memory for encoding stable task-specific priors. The experimental results indicate that the manner in which memory is integrated is as important as the inclusion of memory itself. Extensive empirical evaluation shows that the context-based memory integration strategy adopted in MeCMAT consistently outperforms the gating-based MeGMAT and the layer-based MeLMAT variants, particularly in scenarios requiring long-term coordination. Evaluations on SMAC and GRF further demonstrate that MeCMAT achieves strong performance across the benchmark suite and outperforms representative online baselines in most evaluated scenarios, especially on Super Hard tasks. Ablation studies further suggest that long-term memory and persistent memory play complementary roles in improving long-horizon decision-making. The structured memory framework enables more effective and coordinated policy learning in partially observable multi-agent environments.

In the future, several directions emerge from this work. First, the computational overhead of the memory module, especially in large-scale multi-agent systems, warrants optimization. Second, it is promising to explore adaptive memory allocation, where the capacity of long-term memory or persistent memory can dynamically adjust based on task complexity. Finally, extending this multi-scale memory paradigm to heterogeneous agent teams with distinct roles and observability ranges presents a compelling challenge.

CRediT authorship contribution statement

Gege Sun: Writing – review & editing, Writing – original draft, Visualization, Validation, Methodology, Investigation, Formal analysis, Conceptualization. **Weiqiang Jin:** Writing – review & editing. **Yu Zhang:** Formal analysis. **Guizhong Liu:** Writing – review & editing, Supervision.

Code availability and reproducibility

The implementation of MeMAT is publicly available at https://github.com/suneexxhh/MeMAT. The code version is released as `v1.0`. The repository includes the environment configuration, dependency specifications, training and evaluation scripts, random seed settings, and instructions for reproducing the main results reported in this paper.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix A. Algorithms for MeGMAT and MeLMAT

Algorithm 2 MeGMAT: Multi-agent transformer with gated memory.

Require: Same as Algorithm 1

Initialize Encoder, Decoder, Replay buffer $\mathcal{B}$, long-term memory $\{M_0\}$, persistent memory
for $k = 0, 1, \ldots, K-1$ **do**
  for $t = 0, 1, \ldots, T-1$ **do**
    Collect o_t; generate embeddings x_t
    $\tilde{x}_t = [p_1, \ldots, p_{N_p} \,\|\, x_t]$
    Attention output: $A_t = \mathrm{Attn}(\tilde{x}_t)$
    Memory gate: $G_t = \sigma\big(M_{t-1}(\tilde{x}_t)\big)$
    Gated representation: $y_t = A_t \otimes G_t$ [Eq. (9)]
    Slice $\hat{o}_t$ from y_t; decode actions
    Execute; collect reward; store in $\mathcal{B}$
    Update long-term memory using Eqs. (1)–(3)
  end for
  Training updates same as Algorithm 1.
end for

Algorithm 3 MeLMAT: Multi-agent transformer with memory layer.

Require: Same as Algorithm 1

Initialize Encoder, Decoder, Replay buffer $\mathcal{B}$, long-term memory $\{M_0\}$, persistent memory
for $k = 0, 1, \ldots, K-1$ **do**
  for $t = 0, 1, \ldots, T-1$ **do**
    Collect o_t; generate embeddings x_t
    $\tilde{x}_t = [p_1, \ldots, p_{N_p} \,\|\, x_t]$
    Memory-augment features: $z_t = M_{t-1}(\tilde{x}_t)$
    Compute attention: $y_t = \mathrm{Attn}(z_t)$ [Eq. (10)]
    Slice $\hat{o}_t$ from y_t; decode actions
    Execute; collect reward; store in $\mathcal{B}$
    Update long-term memory using Eqs. (1)–(3)
  end for
  Training updates same as Algorithm 1.
end for

Data availability

The source code used in this study is publicly available at the GitHub repository referenced in the manuscript. No additional proprietary data were generated or analyzed in this study.

References

[1] T. Rashid, M. Samvelyan, C.S. De Witt, G. Farquhar, J. Foerster, S. Whiteson, Monotonic value function factorisation for deep multi-agent reinforcement learning, J. Mach. Learn. Res. 21 (178) (2020) 1–51.
[2] C. Yu, A. Velu, E. Vinitsky, J. Gao, Y. Wang, A. Bayen, Y. Wu, The surprising effectiveness of PPO in cooperative multi-agent games, Adv. Neural Inf. Process. Syst. 35 (2022) 24611–24624.
[3] J.G. Kuba, R. Chen, M. Wen, Y. Wen, F. Sun, J. Wang, Y. Yang, Trust region policy optimisation in multi-agent reinforcement learning, arXiv preprint arXiv:2109.11251, 2021.
[4] M. Wen, J. Kuba, R. Lin, W. Zhang, Y. Wen, J. Wang, Y. Yang, Multi-agent reinforcement learning is a sequence modeling problem, Adv. Neural Inf. Process. Syst. 35 (2022) 16509–16521.
[5] S. Xie, Y. Li, X. Wang, H. Zhang, Z. Zhang, X. Luo, H. Yu, Hierarchical relationship modeling in multi-agent reinforcement learning for mixed cooperative–competitive environments, Inf. Fusion 108 (2024), https://doi.org/10.1016/j.inffus.2024.102318
[6] T. Zhou, F. Zhang, P. Tang, C. Wang, BGC: multi-agent group belief with graph clustering, in: Distributed Artificial Intelligence: Third International Conference, DAI 2021, Shanghai, China, December 17–18, 2021, Proceedings 3, Springer, 2021, pp. 52–63.
[7] R. Zhang, J. Hou, F. Walter, S. Gu, J. Guan, F. Röhrbein, Y. Du, P. Cai, G. Chen, A. Knoll, Multi-agent reinforcement learning for autonomous driving: a survey, arXiv preprint arXiv:2408.09675, 2024.

[8] B. Hegde, M. Bouroche, Multi-agent reinforcement learning for safe lane changes by connected and autonomous vehicles: a survey, AI Commun. 37 (2) (2024) 203–222.

[9] M. Hua, X. Qi, D. Chen, K. Jiang, Z.E. Liu, H. Sun, Q. Zhou, H. Xu, Multi-agent reinforcement learning for connected and automated vehicles control: recent advancements and future prospects, IEEE Trans. Autom. Sci. Eng. (2025), https://doi.org/10.1109/TASE.2025.3574280

[10] J. Shang, S. Meng, J. Hou, X. Zhao, X. Zhou, R. Jiang, L. Qi, Q. Li, Graph-based cooperation multiagent reinforcement learning for intelligent traffic signal control, IEEE Internet Things J. 12 (2025) 14362–14374.

[11] S. Yang, Hierarchical graph multi-agent reinforcement learning for traffic signal control, Inf. Sci. 634 (2023) 55–72, https://doi.org/10.1016/j.ins.2023.03.087

[12] D. Shaheen, G.J.L. Paulraj, I.J. Jebadurai, S. Kirubakaran, Intelligent traffic management system using multi-agent reinforcement learning, in: 2025 International Conference on Electronics and Renewable Systems (ICEARS), IEEE, 2025, pp. 1975–1981.

[13] A. Mushtaq, I.U. Haq, M.A. Sarwar, A. Khan, W. Khalil, M.A. Mughal, Multi-agent reinforcement learning for traffic flow management of autonomous vehicles, Sensors 23 (5) (2023) 2373.

[14] R. Isufaj, M. Omeri, M.A. Piera, Multi-UAV conflict resolution with graph convolutional reinforcement learning, Appl. Sci. 12 (2) (2022), https://doi.org/10.3390/app12020610

[15] Y. Niu, H. Zhao, L. Yu, MA-MIX: value function decomposition for cooperative multiagent reinforcement learning based on multi-head attention mechanism, in: Proceedings of the 23rd International Conference on Autonomous Agents and Multiagent Systems, 2024, pp. 2402–2404.

[16] S. Wang, H. Du, W. Chen, H. Qu, Abbreviated weighted graph in multi-agent reinforcement learning, in: Lecture Notes in Computer Science, Springer, 2024, Ch. 12, pp. 113–124, https://doi.org/10.1007/978-981-99-7019-3_12

[17] C.C. Ekechi, T. Elfouly, A. Alouani, T. Khattab, A survey on UAV control with multi-agent reinforcement learning, Drones 9 (7) (2025) 484.

[18] T. Sun, N. Guo, Y. Cui, Z. Feng, Multi-agent reinforcement learning based on hybrid action representation for UAV swarms' integrated communication and control, IEEE Wirel. Commun. Lett. 15 (2026) 2080–2084.

[19] Z. Wang, Z. Zhang, F. Fang, Y. Du, M3hf: multi-agent reinforcement learning from multi-phase human feedback of mixed quality, arXiv preprint arXiv:2503.02077, 2025.

[20] G. Zhou, Z. Xu, Z. Zhang, G. Fan, Mastering complex coordination through attention-based dynamic graph, in: International Conference on Neural Information Processing, Springer, 2023, pp. 305–318.

[21] M. Mastan, G.J.A. Jose, B. Rakesh, S. Umar, Enhancing hotel recommendations with multi-agent reinforcement learning: a novel MADDPG approach, Expert Syst. Appl. (2026) 131928.

[22] Z. Ning, L. Xie, A survey on multi-agent reinforcement learning and its application, J. Autom. Intell. 3 (2) (2024) 73–91.

[23] J. Liang, H. Miao, K. Li, J. Tan, X. Wang, R. Luo, Y. Jiang, A review of multi-agent reinforcement learning algorithms, Electronics 14 (4) (2025) 820.

[24] A. Beikmohammadi, Learning to communicate through multi-agent reinforcement learning (MARL): a systematic literature review, preprint not peer-reviewed, 2024, https://doi.org/10.20944/preprints202404.0813.v1

[25] M. Kia, J. Cramer, A. Luczak, Memory augmented multi-agent reinforcement learning for cooperative environment, in: International Conference on Artificial Intelligence and Soft Computing, Springer, 2024, pp. 92–103.

[26] E. Pesce, G. Montana, Improving coordination in small-scale multi-agent deep reinforcement learning through memory-driven communication, Mach. Learn. 109 (9) (2020) 1727–1747.

[27] Y. Zhou, D.E. Asher, N.R. Waytowich, J.A. Shah, On memory mechanism in multi-agent reinforcement learning, arXiv preprint arXiv:1909.05232, 2019.

[28] B. Bakker, Reinforcement learning with long short-term memory, in: Advances in Neural Information Processing Systems, 14, MIT Press, 2002, pp. 1475–1482.

[29] Y. Li, X. Wang, J. Wang, W. Wang, X. Luo, S. Xie, Cooperative multi-agent reinforcement learning with hierarchical relation graph under partial observability, in: 2020 IEEE 32nd International Conference on Tools with Artificial Intelligence (ICTAI), IEEE, Baltimore, MD, USA, 2020, pp. 1–8, https://doi.org/10.1109/ictai50040.2020.00011

[30] Y. Yang, G. Chen, W. Wang, X. Hao, J. Hao, P.-A. Heng, Transformer-based working memory for multiagent reinforcement learning with action parsing, Adv. Neural Inf. Process. Syst. 35 (2022) 34874–34886.

[31] A. Baddeley, Working memory: looking back and looking forward, Nat. Rev. Neurosci. 4 (10) (2003) 829–839.

[32] N. Cowan, What are the differences between long-term, short-term, and working memory? Prog. Brain Res. 169 (2008) 323–338.

[33] S. Ahilan, P. Dayan, Feudal multi-agent hierarchies for cooperative reinforcement learning, arXiv preprint arXiv:1901.08492, 2019.

[34] R.J. Williams, D. Zipser, A learning algorithm for continually running fully recurrent neural networks, Neural Comput. 1 (1989) 270–280.

[35] A. Sagirova, Y. Kuratov, M. Burtsev, SRMT: shared memory for multi-agent lifelong pathfinding, arXiv preprint arXiv:2501.13200, 2025.

[36] A. Behrouz, P. Zhong, V. Mirrokni, Titans: learning to memorize at test time, arXiv preprint arXiv:2501.00663, 2024.

[37] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, L. Kaiser, I. Polosukhin, Attention is all you need, Adv. Neural Inf. Process. Syst. 30 (2017).

[38] M. Zhou, Z. Liu, P. Sui, Y. Li, Y.Y. Chung, Learning implicit credit assignment for cooperative multi-agent reinforcement learning, Adv. Neural Inf. Process. Syst. 33 (2020) 11853–11864.

[39] L. Feng, Y. Xie, B. Liu, S. Wang, Multi-level credit assignment for cooperative multi-agent reinforcement learning, Appl. Sci. 12 (14) (2022) 6938.

[40] A.K. Agogino, K. Tumer, Unifying temporal and structural credit assignment problems, in: Autonomous Agents and Multi-Agent Systems Conference, 2004, pp. 121–128.

[41] M. Samvelyan, T. Rashid, C.S. De Witt, G. Farquhar, N. Nardelli, T.G. Rudner, C.-M. Hung, P.H. Torr, J. Foerster, S. Whiteson, The starcraft multi-agent challenge, arXiv preprint arXiv:1902.04043, 2019.

[42] K. Kurach, A. Raichuk, P. Stanczyk, M. Zajac, O. Bachem, L. Espeholt, C. Riquelme, D. Vincent, M. Michalski, O. Bousquet, Google research football: a novel reinforcement learning environment, in: Proceedings of the AAAI Conference on Artificial Intelligence, 34, AAAI Press, 2020, pp. 4501–4510.

[43] M.L. Littman, Markov games as a framework for multi-agent reinforcement learning, in: Machine Learning Proceedings 1994, Morgan Kaufmann, 1994, pp. 157–163.

[44] K. Hornik, M.B. Stinchcombe, H.L. White, Multilayer feedforward networks are universal approximators, Neural Netw. 2 (1989) 359–366.

[45] K. Cho, B. van Merrienboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, Y. Bengio, Learning phrase representations using RNN encoder–decoder for statistical machine translation, in: Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP), Association for Computational Linguistics, 2014, pp. 1724–1734.

[46] G. Mandler, The Structure of Value: Accounting for Taste, Psychology Press, 2014, Ch. 1, pp. 3–36.

[47] S. Morad, R. Kortvelesy, M. Bettini, S. Liwicki, A. Prorok, Popgym: benchmarking partially observable reinforcement learning, arXiv preprint arXiv:2303.01859, 2023.

[48] T. Ni, B. Eysenbach, R. Salakhutdinov, Recurrent model-free RL can be a strong baseline for many POMDPs, in: Proceedings of the 38th International Conference on Machine Learning, PMLR, 2021, pp. 8162–8173.

[49] J. Foerster, G. Farquhar, T. Afouras, N. Nardelli, S. Whiteson, Counterfactual multi-agent policy gradients, in: Proceedings of the AAAI Conference on Artificial Intelligence, 32, AAAI Press, 2018, pp. 2974–2982.

[50] K. Son, D. Kim, W.J. Kang, D.E. Hostallero, Y. Yi, QTRAN: learning to factorize with transformation for cooperative multi-agent reinforcement learning, in: International Conference on Machine Learning, PMLR, 2019, pp. 5887–5896.

[51] J. Wang, Z. Ren, T. Liu, Y. Yu, C. Zhang, QPLEX: duplex dueling multi-agent Q-learning, arXiv preprint arXiv:2008.01062, 2020.

[52] J. Jiang, C. Dun, T. Huang, Z. Lu, Graph convolutional reinforcement learning, arXiv preprint arXiv:1810.09202, 2018.

[53] Q. Yang, W. Dong, Z. Ren, J. Wang, T. Wang, C. Zhang, Self-organized polynomial-time coordination graphs, in: Proceedings of the 39th International Conference on Machine Learning, 162, PMLR, 2022, pp. 24963–24979.

[54] Y. Shi, S. Duan, C. Xu, R. Wang, F. Ye, C. Yuen, Dynamic deep factor graph for multi-agent reinforcement learning, arXiv preprint arXiv:2405.05542, 2024.

[55] M. Tan, Multi-agent reinforcement learning: independent vs. cooperative agents, in: Proceedings of the Tenth International Conference on Machine Learning, 1993, pp. 330–337.

[56] P. Sunehag, G. Lever, A. Gruslys, W.M. Czarnecki, V. Zambaldi, M. Jaderberg, M. Lanctot, N. Sonnerat, J.Z. Leibo, K. Tuyls, Value-decomposition networks for cooperative multi-agent learning, arXiv preprint arXiv:1706.05296, 2017.

[57] Y. Liu, W. Wang, Y. Hu, J. Hao, X. Chen, Y. Gao, Multi-agent game abstraction via graph attention neural network, in: Proceedings of the AAAI Conference on Artificial Intelligence, 34, AAAI Press, 2020, pp. 7211–7218.

[58] W. Boehmer, V. Kurin, S. Whiteson, Deep coordination graphs, in: Proceedings of the 37th International Conference on Machine Learning, 119 of Proceedings of Machine Learning Research, PMLR, 2020, pp. 980–991.

[59] J. Ruan, Y. Du, X. Xiong, D. Xing, X. Li, L. Meng, H. Zhang, J. Wang, B. Xu, Gcs: graph-based coordination strategy for multi-agent reinforcement learning, arXiv preprint arXiv:2201.06257, 2022.

[60] S. Hu, F. Zhu, X. Chang, X. Liang, Updet: universal multi-agent reinforcement learning via policy decoupling with transformers, arXiv preprint arXiv:2101.08001, 2021.

[61] L. Meng, M. Wen, C. Le, X. Li, D. Xing, W. Zhang, Y. Wen, H. Zhang, J. Wang, Y. Yang, B. Xu, Offline pre-trained multi-agent decision transformer, Mach. Intell. Res. 20 (2023) 233–248.

[62] A. Graves, G. Wayne, M. Reynolds, T. Harley, I. Danihelka, A. Grabska-Barwinska, S.G. Colmenarejo, E. Grefenstette, T. Ramalho, J.P. Agapiou, A.P. Badia, K.M. Hermann, Y. Zwols, G. Ostrovski, A. Cain, H. King, C. Summerfield, P. Blunsom, K. Kavukcuoglu, D. Hassabis, Hybrid computing using a neural network with dynamic external memory, Nature 538 (2016) 471–476.

[63] C.-C. Hung, T.P. Lillicrap, J. Abramson, Y. Wu, M. Mirza, F. Carnevale, A. Ahuja, G. Wayne, Optimizing agent behavior over long time scales by transporting value, Nat. Commun. 10 (2018).

[64] T. Ni, M. Ma, B. Eysenbach, P.-L. Bacon, When do transformers shine in RL? Decoupling memory from credit assignment, arXiv preprint arXiv:2307.03864, 2023.

[65] P. Kacham, V.S. Mirrokni, P. Zhong, PolySketchFormer: fast transformers via sketching polynomial kernels, arXiv preprint arXiv:2310.01655, 2024.

[66] A. Katharopoulos, A. Vyas, N. Pappas, F. Fleuret, Transformers are RNNs: fast autoregressive transformers with linear attention, in: Proceedings of the 37th International Conference on Machine Learning, 119 of Proceedings of Machine Learning Research, PMLR, 2020, pp. 5156–5165.

[67] I. Schlag, K. Irie, J. Schmidhuber, Linear transformers are secretly fast weight programmers, in: Proceedings of the 38th International Conference on Machine Learning, 139 of Proceedings of Machine Learning Research, PMLR, 2021, pp. 9355–9366.

[68] R. Loynd, R. Fernandez, A. Celikyilmaz, A. Swaminathan, M. Hausknecht, Working memory graphs, in: International Conference on Machine Learning, PMLR, 2020, pp. 6404–6414.
[69] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, Proximal policy optimization algorithms, arXiv preprint arXiv:1707.06347, 2017.
[70] J. Schulman, P. Moritz, S. Levine, M.I. Jordan, P. Abbeel, High-dimensional continuous control using generalized advantage estimation, arXiv preprint arXiv:1506.02438, 2015.
[71] Y. Du, J.Z. Leibo, U. Islam, R. Willis, P. Sunehag, A review of cooperation in multi-agent learning, arXiv preprint arXiv:2312.05162, 2023.
[72] X. Guo, D. Shi, J. Yu, W. Fan, Heterogeneous multi-agent reinforcement learning for zero-shot scalable collaboration, Neurocomputing (2025) 130716.

Author biography



Gege Sun is currently a Ph.D. candidate in Information and Communication Engineering at Xi'an Jiaotong University, Xi'an, Shaanxi, China. She received a master's degree in Information and Communication Engineering from China Agricultural University. Her research interests include multi-agent reinforcement learning, intelligent decision-making and graph neural networks.



Weiqiang Jin received his M.S. in Computer Science from Shanghai University and is currently pursuing a Ph.D. in Information and Communication Engineering at Xi'an Jiaotong University, China. His main research interests include: 1. LLM-based intelligent reasoning and decision-making, as well as embodied AI; 2. AI for Science (AI4Science). He was a visiting research assistant at The University of Hong Kong from February to September 2025 and is currently a visiting scholar at The Chinese University of Hong Kong under the supervision of Professor Irwin King. His ORCID profile is available at: https://orcid.org/0000-0002-6656-6061, and his Google Scholar can be accessed via: https://scholar.google.com/citations?user=iDZcqAIAAAAJ.



Yu Zhang is currently a Ph.D. candidate in Information and Communication Engineering at Xi'an Jiaotong University, Xi'an, Shaanxi, China. She received a master's degree in Software Engineering from Xi'an Jiaotong University. Her main research interests include multi-agent reinforcement learning, representation learning, and graph reinforcement learning.



Guizhong Liu received the BS and MS degrees in computational mathematics from Xi'an Jiaotong University, Xi'an, Shaanxi, China, in 1982 and 1985, respectively, and the PhD degree in mathematics and computing science from the Eindhoven University of Technology, Eindhoven, The Netherlands, in 1989. He is a full professor with the School of Electronics and Information Engineering, Xi'an Jiaotong University. His research interests include nonstationary signal analysis and processing, image processing, audio and video compression, inversion problems, graph neural networks, embodied AI, visual autonomous agents, visual navigation, and reinforcement learning.